

Hallucinations in LLM-Based Code Summarization: Unveiling, Detection, and Mitigation

GUANGHUA WAN^{*†}, Huazhong University of Science and Technology, China
YUANNING FENG^{*}, Huazhong University of Science and Technology, China
YAO WAN^{†‡}, Huazhong University of Science and Technology, China
ZHAOYANG CHU[†], Huazhong University of Science and Technology, China
ZHANGQIAN BI[†], Huazhong University of Science and Technology, China
JUNXIAO HAN, Hangzhou City University, China
ZHOU ZHAO, Zhejiang University, China
HONGYU ZHANG, Chongqing University, China
PINGPENG YUAN, Huazhong University of Science and Technology, China
XUANHUA SHI[†], Huazhong University of Science and Technology, China
HAI JIN[†], Huazhong University of Science and Technology, China

Code summarization plays an important role in program comprehension and software maintenance by generating natural language descriptions to summarize the semantics of code. While *Large Language Models* (LLMs) have shown remarkable performance in this area, recent empirical studies reveal a critical limitation: *LLMs are prone to hallucinations, producing summaries that are factually inaccurate or unfaithful to the source code, potentially misleading developers*. In this paper, we propose to unveil, detect, and mitigate hallucinations in LLM-based code summarization. First, we construct HALLU-EVAL, a novel dataset for unveiling hallucination phenomena and rigorously evaluating the effectiveness of hallucination detection and mitigation in LLM-based code summarization. It comprises both original code snippets to capture naturally occurring hallucinations and their semantically perturbed counterparts, which are designed to systematically induce challenging logical hallucinations, all complemented with manual hallucination annotations on a curated testbed of 800 code-summary pairs. Next, we propose HALLU-DET, a synergistic approach that combines direct entity-level detection to identify explicit hallucinations with a synonymous mutation-based refinement to reliably confirm or refute more ambiguous cases. Finally, we introduce HALLU-SHIELD, an inference-time mitigation approach

^{*}Equal contribution.

[†]Also with National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, 430074, China.

[‡]Yao Wan is the corresponding author.

Authors' Contact Information: [Guanghua Wan](mailto:wanguanghua@hust.edu.cn), wanguanghua@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China; [Yuanning Feng](mailto:yuanning_feng@hust.edu.cn), yuanning_feng@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China; [Yao Wan](mailto:wanyao@hust.edu.cn), wanyao@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China; [Zhaoyang Chu](mailto:chuzhaoyang@hust.edu.cn), chuzhaoyang@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China; [Zhangqian Bi](mailto:zqbi@hust.edu.cn), zqbi@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China; [Junxiao Han](mailto:hanjx@hzcw.edu.cn), hanjx@hzcw.edu.cn, Hangzhou City University, Hangzhou, China; [Zhou Zhao](mailto:zhaozhou@zju.edu.cn), zhaozhou@zju.edu.cn, Zhejiang University, Hangzhou, China; [Hongyu Zhang](mailto:hyzhang@cqu.edu.cn), hyzhang@cqu.edu.cn, Chongqing University, Chongqing, China; [Pingpeng Yuan](mailto:ppyuan@hust.edu.cn), ppyuan@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China; [Xuanhua Shi](mailto:xhshi@hust.edu.cn), xhshi@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China; [Hai Jin](mailto:hjin@hust.edu.cn), hjin@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE132

<https://doi.org/10.1145/3808139>

that leverages an external value model to guide LLMs toward producing more faithful summaries without costly retraining of the LLM itself. Extensive experiments show that HALLU-EVAL effectively triggers hallucinations, increasing the hallucination rate of models such as Qwen2.5-Coder-7B from 17% to 97% on perturbed code. Our detection approach, HALLU-DET, achieves the best performance among baselines, reaching an F1-score of 0.95 for summaries generated by Qwen2.5-Coder-7B. Moreover, our mitigation method, HALLU-SHIELD, reduces hallucination rates. For example, it lowers the rate from 66% to 59%, a relative reduction of 10.6%, on DeepSeek-Coder-6.7B, while simultaneously improving summary quality.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**.

Additional Key Words and Phrases: Code Summarization, Hallucination, Large Language Models

ACM Reference Format:

Guanghua Wan, Yuanning Feng, Yao Wan, Zhaoyang Chu, Zhangqian Bi, Junxiao Han, Zhou Zhao, Hongyu Zhang, Pingpeng Yuan, Xuanhua Shi, and Hai Jin. 2026. Hallucinations in LLM-Based Code Summarization: Unveiling, Detection, and Mitigation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE132 (July 2026), 24 pages. <https://doi.org/10.1145/3808139>

1 Introduction

Code summarization, which aims to generate natural language descriptions for code snippets, plays a crucial role in program comprehension and software maintenance [11, 19, 43]. The recent success of *Large Language Models* (LLMs) in *Natural Language Processing* (NLP) has spurred substantial advances in code intelligence tasks, including code generation [9, 57] and code summarization [48, 52, 65]. However, there still exists an urgent issue in LLM-based code summarization: *LLMs are prone to generating hallucinations* [20, 27, 50], often producing inaccurate or unfaithful summaries that mislead developers and hinder safe adoption in software development.

A Motivating Example. To better illustrate the hallucinations of LLMs in code summarization, Figure 1 presents a Java code snippet for deleting a snapshot, along with its corresponding summary generated by Qwen2.5-Coder [21], using the prompt in [27] (cf. Section 3.2). From this example, we can observe three common types of hallucinations in code summarization: (1) **Logical Inconsistency**: The summary claims “*If the snapshot parameter is null, the method sets the snapshot to be deleted to null.*” This directly contradicts the actual code logic in line 4, `snapshot != null ? null : snapshot.toString()`, which only processes the snapshot to null if it is not null. This logical inconsistency reveals that the LLM completely misunderstands and inverts the conditional logic. (2) **Factual Incorrectness**: The summary states “*The deleteSnapshot() method is then called on the request object...*”. This is factually incorrect. The source code at line 6, `deleteSnapshot(request)`; shows that `request` is passed as an argument to a method of the current class, not the object on which the method is called. It shows that the LLM confuses the method’s argument with its caller object. (3) **Fabrication of Functionality**: The summary concludes “*...and deletes the corresponding snapshot from the project.*” This claims a successful outcome, namely deleting the snapshot, that is impossible due to a flaw in the code. The logic in line 4 (`snapshot != null ? null : ...`) ensures that a valid snapshot name is never passed to the deletion logic. The LLM completely overlooks this critical flaw and instead fabricates a successful deletion that the code cannot actually perform. Such hallucinations pose a significant risk, as they can severely mislead developers in understanding functions’ actual behavior, which can in turn lead to software maintenance issues such as incorrect feature extensions, wrong API usage, or even the introduction of security vulnerabilities [1, 10].

Our Work. Addressing hallucinations in code summarization presents several key challenges. A primary obstacle is the absence of rigorously curated datasets that expose hallucination issues in LLMs. Although ETF [27], derived from CodeXGLUE [25], has recently been introduced, its samples are relatively simple and fail to uncover more hallucinations rooted in the models. Furthermore,

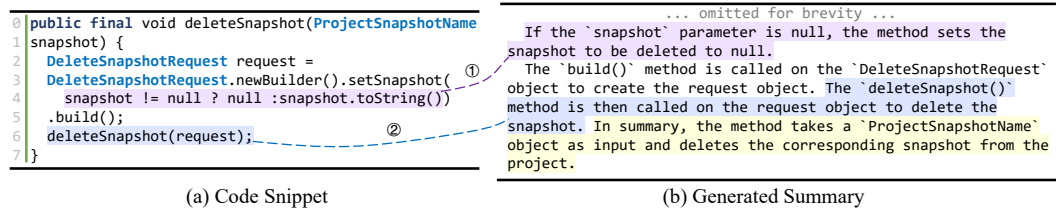


Fig. 1. An example of hallucinated behavior in LLM-generated code summarization. The highlighted sentence describes the hallucinated part and the corresponding code.

existing methods for detection and mitigation are inadequate for the code domain. General-purpose detectors such as SelfCheckGPT [28] do not directly transfer to code summarization because they lack explicit grounding to program semantics (e.g., conditions, control/data flow, and side effects), and may conflate faithful paraphrases/abstractions with errors under brittle lexical matching. Furthermore, powerful mitigation techniques like *Reinforcement Learning from Human Feedback* (RLHF) [35] are often impractical, as they require prohibitively expensive retraining.

In this paper, we propose to unveil, detect and mitigate hallucinations in LLM-based code summarization. We begin by introducing HALLU-EVAL, a challenging dataset designed to induce hallucinated summaries from LLMs by applying semantic-altering perturbations to code. We then present HALLU-DET, which first performs an entity-level hallucination detection for identifying most explicit hallucinations and then conducts a synonymous mutation-based refinement to reliably confirm or refute ambiguous cases. Finally, we propose HALLU-SHIELD, which uses a specialized “guide” model (value model) that evaluates potential next sentences not just for their immediate quality, but for their likelihood of leading to an accurate and faithful summary. During inference, this guide steers the LLM’s decoding process toward generation paths that remain maximally faithful to the code, enhancing LLM-based code summarization reliability.

Key Results and Contributions. We evaluate our HALLU-DET and HALLU-SHIELD approaches on HALLU-EVAL, which proves highly effective at inducing hallucinations by increasing the hallucination rate of Qwen2.5-Coder-7B from 17% on the original code to 97% on our perturbed variants. On this challenging dataset, our detection approach, HALLU-DET, achieves an F1-score of 0.95, significantly outperforming LLM Direct and SelfCheckGPT baselines when applied to detect summaries generated by Qwen2.5-Coder-7B. Furthermore, our mitigation strategy, HALLU-SHIELD, effectively reduces hallucination rates, decreasing the rate for DeepSeek-Coder-6.7B from 66% to 59%, while simultaneously improving summary quality, achieving a 74.0% win rate in comparison with greedy decoding, as judged by an LLM majority-vote ensemble. These results demonstrate HALLU-DET’s good performance in detecting hallucinations in LLM-based code summarization and HALLU-SHIELD’s effectiveness in mitigating them, offering a practical pathway toward more reliable code summarization. The key contributions of this paper are as follows:

- **Systematic Study and New Dataset.** To the best of our knowledge, this is the first systematic study of hallucination in code summarization, covering unveiling, detection, and mitigation. To unveil the hallucination issue and facilitate rigorous evaluation, we introduce HALLU-EVAL, a challenging new dataset built via semantic-altering perturbations, which includes a high-quality, manually annotated subset.
- **Novel Detection Approach.** We propose HALLU-DET, a novel detection approach, which synergizes entity-level detection with a synonymous mutation-based refinement to identify both explicit and ambiguous hallucinations.

- **Novel Mitigation Approach.** We propose HALLU-SHIELD, a value-guided inference-time approach to mitigating the hallucination in LLM-based code summarization. It addresses the short-sightedness of standard decoding by using a value model to steer LLMs toward generating more faithful summaries from a long-term perspective without retraining.

2 Related Work

Code Summarization. Code summarization is a vital task for software maintenance (e.g., maintaining code-comment synchronization [64]) and enhancing program comprehension [15]. It has evolved from early text summarization techniques [18] to neural machine translation approaches [11, 42, 45, 55], alongside rigorous empirical evaluations of these generation techniques [29, 53]. Recently, the emergence of LLMs has garnered significant interest, prompting numerous studies to investigate their potential for code summarization [12, 38, 47, 54]. For example, researchers have investigated diverse prompting techniques, including few-shot prompting to outperform fine-tuned models [2], chain-of-thought to improve reasoning [38], and heuristic questions to guide ChatGPT [47]. Other works have explored knowledge distillation to create smaller, effective models [43], adapter-based knowledge transfer [39], and the applicability of parameter-efficient fine-tuning [4, 46, 56].

Hallucination Detection in LLMs. Existing detection methods can be broadly categorized into two groups. The first verifies LLM outputs against external knowledge sources or the source document itself [7, 30, 31]. In summarization, this verification is often performed at entity level, as the factual consistency of a summary is tied to the accurate representation of its entities and their relationships [27, 32, 41, 62, 66]. The second relies on model-internal signals like token uncertainties [26, 51] and internal states [3, 34, 44], or self-analysis of model outputs via consistency across multiple stochastically sampled outputs [28, 40, 59, 63, 67]. However, general-purpose hallucination detectors are not sufficient for code summarization, where faithfulness hinges on *program-grounded semantics* rather than factual world knowledge. As a result, they may either miss logic-centric inconsistencies or over-flag faithful paraphrases or abstractions as hallucinations. To detect hallucinated code summaries, we therefore follow and extend the entity-level detection paradigm by combining entity-level detection with a synonymous mutation-based refinement.

Hallucination Mitigation. One approach to hallucination mitigation is training-time based, which aims to instill factuality into the model itself through continued training on curated datasets or alignment techniques like RLHF [5, 35] and DPO [13, 23, 36]. While effective at optimizing policies on preference data, these methods are computationally expensive and, critically, cannot be applied to modify pre-existing models. Different from these training-time interventions, a more flexible and practical line of work modifies the generation process at inference-time without retraining. However, existing techniques are often limited to providing context via retrieval-augmented generation [8, 22, 60] or specialized prompting [7], which may not be sufficient to prevent complex logical hallucinations. Inspired by the success of value-guided search [58] in computer vision, our work follows this advanced inference-time concept to the code domain. We propose an effective strategy to reduce hallucinations in code summaries, using our specialized HALLU-DET detector as the reward signal to directly optimize for the factual correctness and faithfulness unique to code.

3 Unveiling Hallucinations in Code Summarization

We first introduce HALLU-EVAL, a challenging new dataset specifically constructed to unveil hallucinations in code summarization. This dataset addresses a critical challenge: existing datasets for hallucination in code summarization largely contain standard programming patterns, which fall short of sufficiently evaluating the advanced semantic reasoning capabilities of LLMs. Figure 2

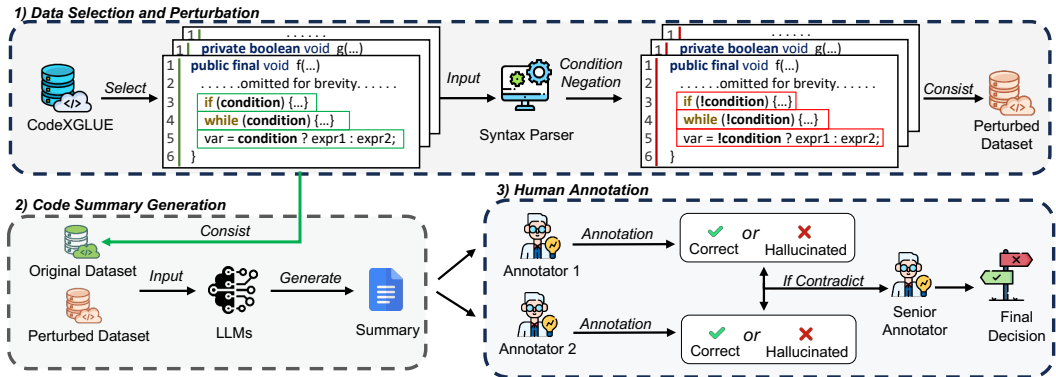


Fig. 2. The pipeline of HALLU-EVAL dataset construction.

presents the pipeline of constructing our dataset, which comprises two phases: first, generating a large corpus of (code, summary) pairs via semantic perturbation; and then, curating a high-quality, manually annotated subset to facilitate the evaluation of hallucination detection. This dataset serves as the foundation for the detection and mitigation techniques discussed subsequently.

3.1 Data Selection and Perturbation

We begin with Java methods from the CodeXGLUE [25] code-to-text subset, filtering 10,000 code snippets that contain conditional logic. We focus on conditional code because conditional constructs are a primary locus of semantic brittleness in code summarization: many high-impact faithfulness failures stem from misstating branch conditions, null-handling guards, or control-flow behavior, where subtle predicate changes can invert program semantics while remaining superficially plausible. This filtering step thus defines the scope of our study toward logic-centric faithfulness, rather than aiming to maximize hallucination rates by cherry-picking samples.

However, our preliminary analysis reveals that these initial samples predominantly contain standard programming patterns, where the overall hallucination rate is relatively low. In particular, the incidence of the most challenging hallucination type, *logical inconsistency*, which requires the model to correctly interpret complex program logic and control flow, is limited. This motivates us to propose a more targeted approach to augmenting the dataset.

To achieve this, we explore multiple perturbation strategies designed to systematically unveil hallucinations. Through a preliminary analysis on 50 pilot samples, we find that non-semantic perturbations (e.g., dead code, and variable renaming) are ineffective (< 18% hallucination rate), while among semantic strategies (e.g., altering operators, changing constant values, and inverting conditional logic), *inverting conditional logic* via predicate negation proves uniquely effective (~ 98% induction rate). Consequently, we select this approach to create a dense set of challenging tests that precisely isolate a model's semantic reasoning capabilities. We implement this via static analysis, parsing the *Abstract Syntax Tree* (AST) to apply a *Predicate Negation Operator* (PNO) that inverts the logic of conditional constructs (e.g., transforming `a==1 && b==2` to `a!=1 || b!=2`), as illustrated in Figure 3. Note that PNO may yield code whose control-flow is intentionally degenerate (e.g., loop negation). This is by design, because our goal is to invert semantics to stress logical faithfulness, not to preserve original behavior. To ensure dataset diversity and comprehensive evaluation, we retain both the original, unperturbed code snippets and their logically inverted counterparts.

Dataset Naming. This process yields two dataset components: HALLU-EVAL (Easy), the original code, and HALLU-EVAL (Hard), the challenging adversarially perturbed variants.

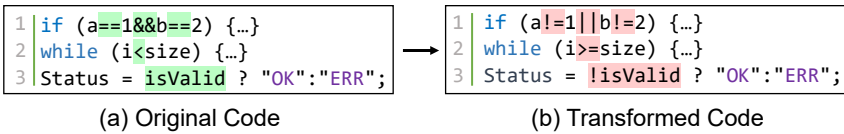


Fig. 3. Predicate negation of conditional constructs.

3.2 Code Summary Generation

With the original and perturbed code snippets in place, we prompt LLMs to generate a diverse corpus of summaries for analysis. Specifically, we randomly select 300 code snippets from HALLU-EVAL (Easy) and their corresponding variants in HALLU-EVAL (Hard). To ensure structural representativeness, we verify that the Halstead complexity distribution of these selected snippets closely aligns with the full CodeXGLUE corpus. We then employ four code LLMs of varying sizes that show good performance in code summarization: Qwen2.5-Coder-7B-Instruct, Qwen2.5-Coder-14B-Instruct [21], CodeLlama-7B-Instruct, and CodeLlama-13B-Instruct [37]. For each selected snippet, each of the four models is prompted using a standardized instruction format, as follows:

Prompts for Code Summary Generation

As a Java Expert, provide a detailed summary of the following Java code. Please structure your summary into the following sections:

1. Inputs and outputs of the method.
2. Business purpose.
3. Detailed functional summary of the method (You have to explain the code line by line).

{CODE}

Finally, we obtain a total of 2,400 (code variant, summary) pairs, enabling controlled evaluation of an LLM's ability to generate faithful summaries, particularly in the presence of subtle but semantically important code changes.

3.3 Human Annotation

We select a significant and representative subset of the generated summaries for a meticulous human annotation process. This subset consists of 800 (code, summary) pairs, derived from outputs generated by four LLMs on 100 code snippets from HALLU-EVAL (Easy) and their corresponding counterparts from HALLU-EVAL (Hard). This sample size ensures a statistically robust margin of error ($< 4\%$ at 95% confidence). The annotation is conducted by two independent annotators, both graduate students in computer science with over five years of experience in Java programming and software development. For each (code, summary) pair, annotators are instructed to assess whether the summary exhibits any form of hallucination. This includes identifying factual inaccuracies, misrepresentations of control flow (particularly conditional logic), or the presence of ungrounded entities or behaviors not evidenced in the code. Based on this evaluation, each summary is assigned a binary label: "Hallucinated" or "Correct". All 800 pairs are reviewed independently by both annotators, yielding a Cohen's Kappa score of 0.77, which indicates substantial inter-annotator agreement. In cases of disagreement, a third senior annotator, an experienced researcher in software engineering, facilitates a deliberation process to resolve conflicts and determine the final consensus label for each instance. This annotation process results in a high-quality testbed of 800 (code, summary, label) triplets, which serves as the ground truth for our evaluation. All subsequent experiments are conducted exclusively on this 800-pair annotated subset.

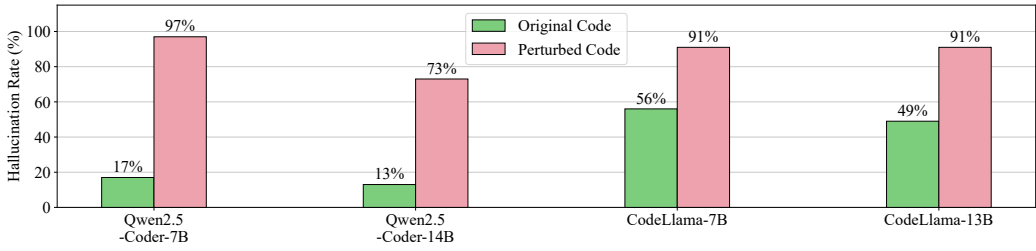


Fig. 4. Hallucination rates on HALLU-EVAL (Easy) vs. HALLU-EVAL (Hard).

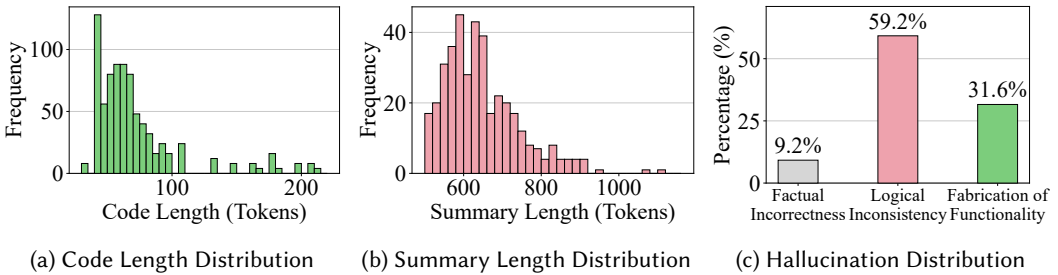


Fig. 5. Detailed characteristics of the HALLU-EVAL annotated dataset.

3.4 Dataset Analysis

We now conduct a pilot study to validate HALLU-EVAL's core purpose: its ability to systematically induce and expose hallucinations in LLMs. As shown in Figure 4, on the original code samples (HALLU-EVAL (Easy)), LLMs already exhibit a non-trivial hallucination rate. However, on the perturbed variants (HALLU-EVAL (Hard)), this rate surges dramatically. For instance, escalating from 17% to 97% for Qwen2.5-Coder-7B. This demonstrates that even powerful models struggle with our dataset, as our perturbations create a direct conflict between the code's superficial textual patterns and its altered execution logic, confirming that HALLU-EVAL serves as a rigorous testbed for evaluating hallucination in code summarization.

This effectiveness stems from the dataset's specific design, detailed in Figure 5. By prompting for verbose, multi-step summaries (Figure 5b) for relatively concise code snippets (Figure 5a), we create a demanding scenario which can make hallucinations more salient. The distribution of these hallucinations (Figure 5c), as determined by our manual human annotation process, is a direct outcome of our perturbation strategy: logical inconsistency emerges as the predominant failure mode (59.2%), confirming that the primary challenge posed by HALLU-EVAL is not surface-level grounding but deep semantic comprehension.

4 HALLU-DET: A Hallucination Detector

Following the principle of entity-level verification [27, 32, 62] in summarization tasks, we propose HALLU-DET, a novel approach that synergizes direct entity-level detection with a synonymous mutation-based refinement, as shown in Figure 6. From our investigation, we find that while this initial entity-level detection performs well in identifying explicit hallucinations, it can be inconclusive when faced with cases where the phrasing is ambiguous.

To resolve these ambiguous cases and ensure high precision, we further refine the entity-level detection with a synonymous mutation-based re-evaluation. The key idea is that this refinement performs a deeper semantic check: a genuinely flawed summary is semantically fragile and will likely be exposed as inconsistent when rephrased, whereas a factually correct statement will

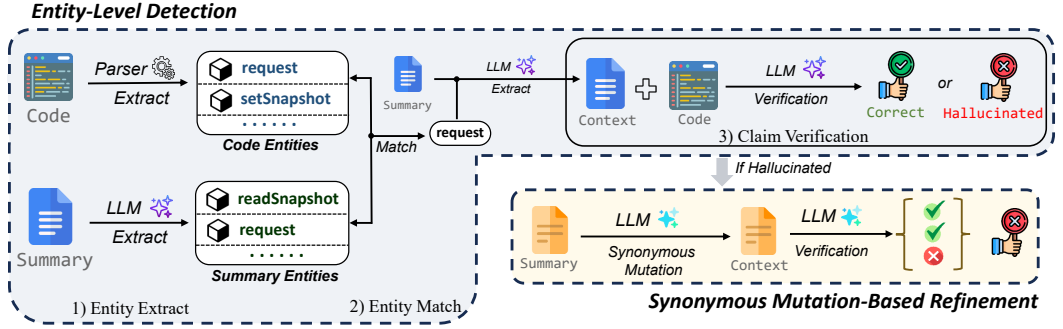


Fig. 6. An overview of the hallucination detector HALLU-DET.

remain consistent across multiple synonymous mutations. This allows us to definitively confirm hallucinations in ambiguous cases.

4.1 Entity-Level Detection

The entity-level detection component of HALLU-DET verifies a summary’s claims against the verifiable information in the source code. In particular, we first perform entity extraction and matching to identify entities present in the summary and link them to the source code. Subsequently, we conduct claim verification on these matched entities to assess whether the summary’s description of their purpose and behavior is consistent with the code’s actual logic.

Entity Extraction and Matching. We begin by extracting entities (e.g., function names, variable names, class names, literal values) from two sources: the source code and the generated summary. For the source code, we traverse its AST using static analysis tools (i.e., Javalang) to extract entities from their corresponding node types. For the summary, we prompt an LLM (e.g., Gemini-2.5-flash) with a direct instruction to “extract all code-related entities mentioned in the text and list them”. Following extraction, we match the entities from the summary to the entities from the code using string-based matching heuristics. This process categorizes each summary entity as either a “MATCH” or “MISMATCH”. Entities labeled “MATCH” are subject to claim verification. For entities labeled “MISMATCH”, we do not flag them as hallucinations. We adopt this conservative strategy to avoid the critical pitfall of overly-strict matching, which often leads to false positives due to paraphrasing, abstract references (e.g., referring to a specific class as ‘the object’), naming convention differences, or occasional LLM omissions during extraction. By ignoring mismatches, we prioritize high precision, ensuring that we only penalize explicit contradictions rather than missing or loosely referenced entities.

Claim Verification. For each successfully matched entity, we verify the claims made about it in the summary. We first extract the relevant “claim context”, which consists of all statements within the summary that describe the entity’s purpose and behavior. A key challenge here is that a naive context extraction method, such as simple string matching, is insufficient because it fails to capture complex linguistic phenomena. For example, it can easily miss co-references (where a pronoun like “it” refers to an entity mentioned earlier) or descriptions split across multiple sentences. To address this limitation, we prompt a verification LLM (e.g., Gemini-2.5-flash) with the code snippet and the entity, instructing it to extract all sentences from the summary that describe the entity’s behavior, role, or interactions, including resolving any co-references. This complete (code entity, summary context) tuple is then passed to a verification LLM (e.g., Gemini-2.5-flash), which assesses consistency between the code’s behavior and the summary’s claims. This semantic

analysis is particularly crucial for detecting *logical inconsistency*. Unlike basic entity linking which only validates existence, extracting the full claim context allows the verifier to interpret conditional statements (e.g., ‘if x is null’) and logical predicates. This enables HALLU-DET to identify cases where the summary’s narrative contradicts the code’s control flow, even when all entities are correctly named. If the summary context is found to be inconsistent, it is flagged as “HALLUCINATED”.

4.2 Synonymous Mutation-Based Refinement

This refinement process is designed to resolve the ambiguous cases and potential false positives identified during the initial entity-level detection. The key insight is that a true hallucination is semantically fragile, while a correct but nuanced statement is robust [28, 63]. A genuinely flawed summary segment will likely be exposed as inconsistent when rephrased, whereas a valid statement will remain factually correct and faithful across multiple synonymous mutations.

Synonymous Mutation Generation. Given a summary segment flagged as potentially hallucinated by the direct entity-level detection, we prompt an LLM (e.g., Gemini-2.5-flash) to generate N semantically equivalent but lexically distinct variants, which we refer to as synonymous mutations. This step aims to obtain multiple alternative expressions of the same core claim.

Re-evaluation of Mutations. Each of the N generated mutations is independently evaluated for factual correctness and faithfulness against the original source code. For each mutation, we prompt a verifier LLM (e.g., Gemini-2.5-flash) to make a holistic judgment on whether the summary contains hallucinations against the code.

Refined Decision Logic. Based on the mutation verification results, we make a final decision for the originally flagged segment. If all N mutations are judged “Accurate”, we override the initial finding and classify the segment as “Correct”. Conversely, if any mutation is determined to be “inaccurate”, we confirm the original suspicion and retain the hallucination label.

5 HALLU-SHIELD: A Reinforcement Learning Approach for Hallucination Mitigation

We introduce HALLU-SHIELD, a mitigation approach that guides LLMs to produce more faithful summaries by looking ahead, rather than just optimizing for the most plausible next words. Crucially, HALLU-SHIELD is intricately coupled with HALLU-DET. Instead of relying on expensive human feedback, HALLU-DET provides automated supervision signals (reward scores), allowing HALLU-SHIELD to bootstrap its mitigation capabilities. This self-supervised design eliminates the manual annotation bottleneck, ensuring practical applicability.

5.1 From Short-Sighted Generation to a Far-Sighted Strategy

To understand our approach, we must first briefly introduce how LMs generate text and the inherent limitations of the standard method.

Language Models and Autoregressive Generation. At their core, LLMs like the ones used in this paper are autoregressive generators. Given a sequence of tokens $s = (w_1, w_2, \dots, w_T)$, they model the joint probability of the sequence using the chain rule of probability:

$$p(s) = \prod_{t=1}^T p(w_t | w_1, \dots, w_{t-1}). \quad (1)$$

A neural network parameterized by θ is trained to model these conditional probabilities $p(w_t | w_{<t}; \theta)$. The standard training objective is maximum likelihood estimation, where the model’s parameters θ are optimized to maximize the log-probability of the ground-truth summaries in the training data.

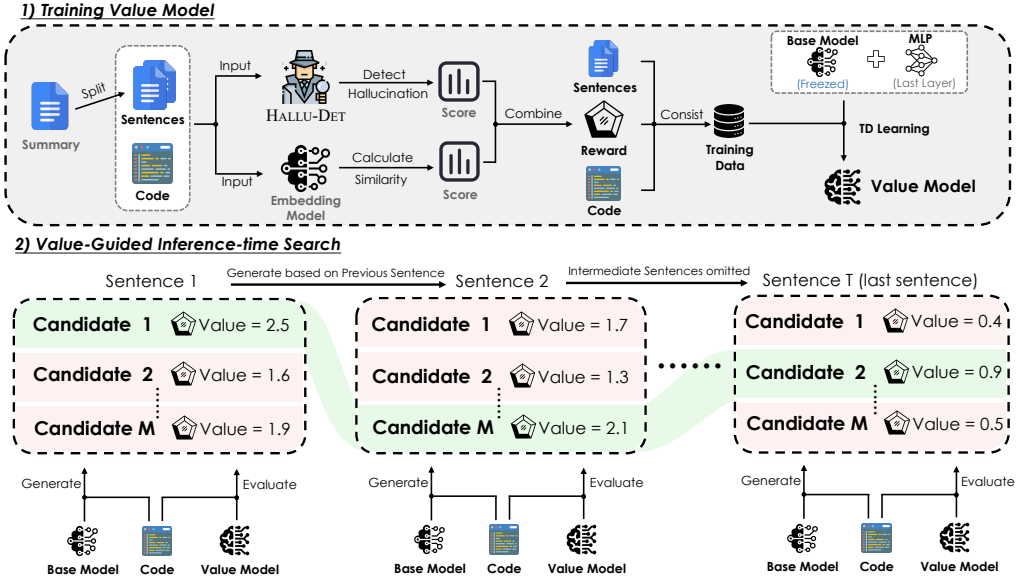


Fig. 7. The pipeline of HALLU-SHIELD, our value-guided inference-time mitigation approach.

The Default Strategy: Greedy Decoding and its Limitations. Once the model θ is trained, a decoding strategy is required to generate a new summary one token at a time. The most common strategy is greedy decoding. At each step t , the algorithm simply selects the single token with the highest conditional probability:

$$w_t = \arg \max_{w \in V} p(w \mid w_{<t}; \theta). \quad (2)$$

This equation formalizes the greedy approach. At each generation step t , the next token w_t is chosen by selecting the token w from the entire vocabulary V that maximizes the model's predicted probability, given the preceding tokens $w_{<t}$. The critical flaw revealed here is its short-sightedness: the *argmax* optimization is performed independently at each step, selecting the best immediate next token without considering the quality of the overall sequence that this choice will produce.

Summarization as an MDP. To overcome the short-sightedness of greedy decoding, we first re-frame the task of writing a summary not as a single action, but as a sequence of decisions. At each step, the model must decide which sentence to write next, and each decision impacts the quality and faithfulness of the final summary. This step-by-step process can be formally modeled as a *Markov Decision Process* (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. In our code summarization context, these formal components map directly to the generation task: the state $s_t \in \mathcal{S}$ is the partial summary that has been written so far, an action $a_t \in \mathcal{A}$ is the act of generating the next candidate sentence, and the reward $r_t \in R$ is an immediate score measuring the faithfulness of that new sentence. The goal is thus to find a sequence of actions (sentences) that maximizes the total cumulative reward (the sum of all rewards for all sentences), expressed as $\sum_{k=0}^{\infty} \gamma^k r_{t+k}$, where γ is a discount factor. This formally distinguishes our “far-sighted” goal from a “short-sighted” one that only considers the immediate reward r_t .

Figure 7 shows an overview of the proposed HALLU-SHIELD for code summarization, when generating a summary for a given source code snippet, at each step of producing a new sentence, the base LLM generates multiple candidate sentences. Our trained value model, HALLU-VALUE, then evaluates these candidates for their potential to lead to a complete summary that is faithful and

free of hallucinations (i.e., their long-term value). We then use this value-based guidance to select the optimal sentence to append to the ongoing summary.

5.2 Reward and Value Model for Code Summaries

The cornerstone of our mitigation strategy is the reward function, R , which provides the ground-truth signal for faithfulness. We define the immediate reward $R(s_i)$ for a sentence s_i as a composite score reflecting two key aspects:

Absence of Hallucination in the Sentence: Determined by HALLU-DET, it provides a hallucination decision for each sentence. We transform this into a reward component $r_h(s_i)$:

$$r_h(s_i) = \begin{cases} 0 & \text{if hallucinated in } s_i, \\ 1 & \text{otherwise.} \end{cases}$$

Semantic Relevance of the Sentence to the Source Code: We compute the semantic similarity score between the sentence s_i and a representation of the entire source code snippet C , using jina-embeddings-v2-base-code model [16], an embedding model specifically optimized for code understanding and semantic search. We denote this semantic similarity score as $r_{sim}(s_i)$.

The final reward is computed as $R(s_i) = r_h(s_i) \cdot r_{sim}(s_i)$. This multiplicative formulation ensures that if a sentence s_i contains a hallucination, its reward is immediately reduced to zero, strongly penalizing any factual deviation.

While this immediate reward is useful for evaluation, it is impossible to know the full cumulative reward during generation. We therefore introduce our specialized value model, HALLU-VALUE, which is trained to predict this long-term value from any given state. The objective of HALLU-VALUE is to learn a value function, $V(S_t, C)$, which estimates the expected cumulative reward for a partial summary S_t and source code C . We train HALLU-VALUE with Temporal Difference (TD) learning, where at step t (generating sentence s_t from S_{t-1}), the value of the previous state S_{t-1} is updated to match the immediate reward for s_t plus the discounted value of the resulting state S_t . Formally,

$$V(S_{t-1}, C) \approx \mathbb{E}[R(s_t) + \gamma V(S_t, C)] . \quad (3)$$

This process allows HALLU-VALUE to learn which plausible-sounding initial sentences are actually “dead ends” that are likely to lead to future hallucinations.

5.3 Value-Guided Summary Generation

We employ the trained HALLU-VALUE during the inference phase to dynamically guide the base LLM towards generating more factual and faithful summaries. At each step t of generating the summary (where a step corresponds to generating a new sentence s_t), our process involves exploring multiple candidate sentences and selecting the one with the highest predicted long-term value.

Specifically, when the base LLM generates the sentence s_t at step t based on the previously generated partial summary $S_{t-1} = s_1, \dots, s_{t-1}$, we first generate a set of M candidate next sentences $\{s'_{t,1}, s'_{t,2}, \dots, s'_{t,M}\}$. We achieve this by prompting the base LLM multiple times using stochastic decoding strategies, such as employing a list of different temperature settings $T = \{T_1, \dots, T_N\}$ and performing K sampling operations for each temperature (thus $M = N \times K$). For each of these M candidate sentences $s'_{t,j}$, our HALLU-VALUE predicts its long-term value $V(S_{t-1} + s'_{t,j}, C)$. This value represents the expected cumulative reward if the summary is continued from this point by appending $s'_{t,j}$.

Among the M candidates, we select the one, denoted as s_t^* that maximizes this predicted long-term value:

$$s_t^* = \operatorname{argmax}_{s'_{t,j} \in \{s'_{t,1}, \dots, s'_{t,M}\}} V(S_{t-1} + s'_{t,j}, C). \quad (4)$$

This selected sentence s_t^* is then appended to the current partial summary S_{t-1} to form $S_t = S_{t-1} + s_t^*$. This iterative process of candidate generation, value-based evaluation, and selection continues until a complete summary is formed (e.g., an end-of-summary token is generated by a candidate sentence or a maximum length is reached). This dynamic, sentence-level intervention, guided by the forward-looking predictions of HALLU-VALUE, allows us to steer the generation towards sequences that are not only locally plausible but are also anticipated to result in a globally more faithful and higher-quality summary, thereby mitigating hallucinations.

6 Experiments and Analysis

We conduct experiments to rigorously evaluate our proposed hallucination detection framework and mitigation method. Our analysis is structured around the following *Research Questions* (RQs):

▷ **Study I: Hallucination Detection Performance.** In this study, we evaluate the effectiveness, composition, and robustness of our detection approach, HALLU-DET.

- **RQ1-1: Overall Detection Performance.** How effective is our HALLU-DET approach in detecting hallucinations in code summaries, and how does its performance compare against detection baselines on our challenging HALLU-EVAL dataset?
- **RQ1-2: Component Contribution Analysis.** What are the respective contributions of the direct entity-level detection and synonymous mutation-based refinement components to the overall performance of HALLU-DET?
- **RQ1-3: Robustness and Efficiency.** How robust is the detection performance of HALLU-DET across different hallucination categories, and what is its computational efficiency?

▷ **Study II: Hallucination Mitigation Effectiveness.** In this study, we evaluate the effectiveness, robustness, and practical efficiency of our mitigation approach, HALLU-SHIELD.

- **RQ2-1: Overall Mitigation Effectiveness.** How effective is HALLU-SHIELD in producing summaries that have fewer hallucinations and are of higher overall quality than baseline mitigation techniques?
- **RQ2-2: Robustness Across Hallucination Categories.** How robust is HALLU-SHIELD's mitigation performance across different hallucination categories (factual incorrectness, logical inconsistency, and fabrication of functionality)?
- **RQ2-3: Ablation and Efficiency Analysis.** What are the key drivers of HALLU-SHIELD's effectiveness, and what are its practical trade-offs in terms of computational overhead?

6.1 Experimental Setup

6.1.1 Studied Models. We adopt four widely used LLMs for code summarization with different model sizes:

- **Qwen2.5-Coder** [21] released by the Qwen team, is a series of code-specific LLMs built upon the Qwen2.5 architecture. Our experiments use the 7B-Instruct and 14B-Instruct versions.
- **CodeLlama** [37] released by Meta AI, is further trained on Llama 2 with 0.5 trillion code-specific tokens. Our experiments use the 7B-Instruct and 13B-Instruct versions.
- **DeepSeek-Coder** [17] released by DeepSeek AI, is a series of code-specialist LLMs trained from scratch on a massive 2-trillion-token dataset. Our experiments use the 6.7B-Instruct version.
- **Llama3** [24] released by Meta. Llama 3 models are pretrained on an extensive, custom-curated 15-trillion-token dataset. We employ the 8B-Instruct version in our experiments.

6.1.2 Baselines. To evaluate the effectiveness of our HALLU-DET detection framework and HALLU-SHIELD mitigating strategy, we compare them with the following baselines:

Hallucination Detection. We compare HALLU-DET against the following three baselines.

- **LLM Direct:** We prompt an LLM (i.e., Gemini-2.5-flash [14]) to directly assess whether a given (code, summary) pair contains hallucinations, outputting a binary judgment.
- **ETF [27]:** A recent baseline focusing on entity-level verification for code summarization. It relies on string-matching heuristics to extract and compare entities, rigidly labeling any mismatched entity between the source code and the generated summary as a hallucination.
- **SelfCheckGPT [28]:** A zero-resource black-box method in NLP. We use original LLM to generate multiple alternative summaries for the same code via stochastic sampling. The consistency of the original summary's sentences against these alternatives is then measured; low consistency suggests potential hallucination. We follow the threshold setting in [28].

Hallucination Mitigation. We compare HALLU-SHIELD against four strong baselines. As no established methods exist specifically for hallucination mitigation in code summarization, we adapt several prominent prompting and decoding strategies for this task:

- **Greedy Decoding/Default:** The summaries generated directly by the base LLMs without any mitigation strategy, representing the default performance.
- **Factuality Prompting:** We enhance the input prompt for the base LLM with explicit instructions aimed at improving factual consistency. The prompt directs the model to strictly adhere to the provided code's logic and avoid inferring functionalities not explicitly present.
- **Chain-of-Thought (CoT):** We adopt the CoT prompting technique to code summarization. The LLM is first instructed to generate a step-by-step analysis of the code's logic, including its variables, control flow, and operations. It is then prompted to produce the final summary based only on its preceding analysis.
- **Best-of-N (BoN) Sampling:** For each code snippet, the base LLM generates N candidates with the same temperature set used by HALLU-SHIELD, and an external LLM (Qwen2.5-Coder-7B) ranks them to select the most faithful, high-quality summary.

6.1.3 Implementation Details. For our HALLU-DET detection framework, we use Gemini-2.5-flash and Qwen3-32B for all LLM-based judgment tasks and generate $N = 6$ synonymous mutations in the re-evaluation stage. To mitigate the inherent randomness of LLM generations and reduce evaluation bias (e.g., generation variations even at a temperature setting of 0), all our experiments were repeated three times, and the average results are reported. For the HALLU-SHIELD mitigation strategy, the HALLU-VALUE model is based on the Qwen2.5-Coder-7B and is finetuned for 3 epochs on 5,000 samples with a learning rate of $5e-6$, a batch size of 256, and the Adam optimizer. During its value-guided inference, we generate 6 candidate sentences at each step with temperatures of 0, 0.1, 0.3, 0.5, 0.7, and 0.9. To ensure a fair comparison, the Best-of-N (BoN) baseline is also configured to generate and rank a total of $N = 6$ candidate summaries using the same set of temperatures. All experiments are conducted on a server with NVIDIA H800 GPUs. The HALLU-VALUE model training is performed on seven NVIDIA H800 GPUs for approximately six hours, until the loss converges.

6.2 Hallucination Detection Performance (Study I)

6.2.1 RQ1-1: Overall Detection Performance. The comprehensive results are presented in Table 1. We can observe that in terms of metrics such as F1, Acc, and R, HALLU-DET consistently achieves the best performance compared to the baseline methods, demonstrating superior F1-scores across all models and scenarios. While LLM Direct occasionally achieves competitive Precision, HALLU-DET significantly excels in Recall, a paramount metric in software engineering, as missing a

Table 1. Performance comparison of hallucination detection methods on HALLU-EVAL, evaluated across different target LLMs and detector backbones. F1: F1-Score, Acc: Accuracy, P: Precision, R: Recall.

	HALLU-DET				LLM Direct				ETF				SelfCheckGPT			
	F1	Acc	P	R	F1	Acc	P	R	F1	Acc	P	R	F1	Acc	P	R
Detector Backbone: Gemini-2.5-Flash																
Qwen2.5-Coder-7B	0.95	0.94	0.92	0.98	0.91	0.91	0.95	0.88	0.76	0.67	0.64	0.94	0.55	0.49	0.55	0.54
Qwen2.5-Coder-14B	0.88	0.89	0.81	0.97	0.82	0.86	0.90	0.76	0.66	0.58	0.51	0.94	0.46	0.40	0.37	0.60
CodeLlama-7B	0.94	0.91	0.92	0.97	0.93	0.90	0.93	0.94	0.82	0.72	0.77	0.89	0.34	0.37	0.73	0.22
CodeLlama-13B	0.91	0.87	0.88	0.93	0.90	0.86	0.90	0.90	0.82	0.73	0.76	0.88	0.25	0.36	0.67	0.16
Detector Backbone: Qwen3-32B																
Qwen2.5-Coder-7B	0.92	0.92	0.99	0.86	0.89	0.89	0.99	0.81	0.77	0.70	0.67	0.91	0.55	0.49	0.55	0.54
Qwen2.5-Coder-14B	0.77	0.82	0.87	0.69	0.74	0.80	0.81	0.69	0.66	0.59	0.51	0.94	0.46	0.40	0.37	0.60
CodeLlama-7B	0.88	0.84	0.92	0.85	0.77	0.71	0.90	0.67	0.85	0.75	0.75	0.98	0.34	0.37	0.73	0.22
CodeLlama-13B	0.83	0.77	0.86	0.79	0.79	0.75	0.93	0.69	0.82	0.70	0.70	0.99	0.25	0.36	0.67	0.16

hallucinated summary (false negative) can mislead developers and introduce severe bugs. This superior performance is demonstrated across the entire HALLU-EVAL dataset, which includes both original code samples with naturally occurring hallucinations and perturbed samples with induced logical inconsistency. For example, when evaluating summaries from Qwen2.5-Coder-14B, HALLU-DET achieves an impressive F1-score of 0.88, significantly outperforming the LLM Direct baseline (0.82) and trouncing the consistency-based SelfCheckGPT (0.46). Moreover, it surpasses the recent entity-based baseline ETF (0.66) by a wide margin. This substantial performance gap highlights the limitations of ETF's string-matching heuristics in capturing complex logical dependencies, which our LLM-based semantic verification successfully addresses. Furthermore, we additionally validate with Qwen3-32B as an open-source backbone as shown in Table 1, where HALLU-DET consistently outperforms all baselines, confirming its robustness across detector engines. This improvement is consistent across all four models, reflecting the superiority of our method.

📖 **Answer to RQ1-1:** The HALLU-DET detection framework is highly effective, significantly outperforming baselines in hallucination detection and demonstrating strong robustness across different detector backbones.

6.2.2 RQ1-2: Component Contribution Analysis. To understand the contribution of each component in our HALLU-DET, we conduct an ablation study. Table 2 compares the performance of the entity-level detection component when used independently against the full HALLU-DET approach. We observe that this initial detection step provides a strong performance, achieving a high Recall of 0.98 for summaries from Qwen2.5-Coder-7B as it effectively catches explicit factual hallucinations. However, its precision is lower (0.90), resulting in a slightly suppressed F1-score of 0.94. This lower precision indicates a notable rate of false positives, where semantically correct but nuanced phrasing is incorrectly flagged. After incorporating the synonymous mutation-based refinement, the full HALLU-DET approach achieves a comprehensive improvement. While Recall remains high at 0.98, precision increases significantly to 0.92, boosting the overall F1-score to 0.95. While this increment appears modest, it is primarily because our initial entity-level detection with the proprietary Gemini-2.5-flash already achieves near-ceiling performance. The refinement component acts as a strict filter for ambiguous cases by exploiting the “semantic fragility” of hallucinations to minimize false positives. To decisively validate its necessity, we conduct an additional ablation using an open-source backbone (Qwen3-32B). In this setting, the mutation refinement significantly improves average precision by approximately 27% without sacrificing recall, confirming its critical role in ensuring high precision for practical deployments.

Table 2. HALLU-DET detection performance on HALLU-EVAL (HALLU-DET vs. Entity-Level Detection vs. Mutation Refinement (refers to Synonymous Mutation-based Refinement)).

	HALLU-DET				Entity-Level Detection				Mutation Refinement			
	F1	Acc	P	R	F1	Acc	P	R	F1	Acc	P	R
Qwen2.5-Coder-7B	0.95	0.94	0.92	0.98	0.94	0.93	0.90	0.98	0.74	0.60	0.59	1.00
Qwen2.5-Coder-14B	0.88	0.89	0.81	0.97	0.87	0.88	0.79	0.97	0.61	0.45	0.44	1.00
CodeLlama-7B	0.94	0.91	0.92	0.97	0.94	0.91	0.92	0.97	0.85	0.74	0.74	1.00
CodeLlama-13B	0.91	0.87	0.88	0.93	0.91	0.87	0.88	0.93	0.83	0.71	0.70	1.00

📖 **Answer to RQ1-2:** Our analysis confirms that the synergistic design of HALLU-DET is critical to its effectiveness.

6.2.3 *RQ1-3: Robustness and Efficiency Analysis.* We analyze HALLU-DET from two perspectives: robustness across hallucination categories and computational efficiency.

Robustness Across Hallucination Categories. To provide a more granular understanding of its overall performance, we analyze HALLU-DET's detection accuracy across the three primary hallucination types identified in our motivating example, relying on the ground truth established by our rigorous manual human annotations. As illustrated in Figure 8, the results demonstrate that HALLU-DET is not only highly effective in aggregate but also remarkably robust and versatile. It achieves consistently high accuracy across all error categories, successfully identifying 97.7% of factual incorrectness, 96.2% of logical inconsistency, and 95.3% of fabrication of functionality instances. This finding validates that our HALLU-DET approach is highly effective at uncovering all types of hallucinations in LLM-based code summarization.

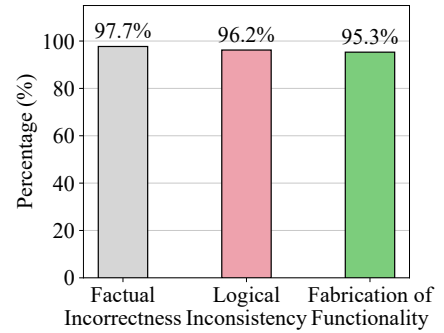


Fig. 8. Detection accuracy across hallucination categories.

Efficiency Analysis. Beyond detection accuracy, we evaluate computational efficiency to address deployment concerns. Our pipeline is designed with a streamlined hierarchical structure. In contrast to the LLM Direct baseline, which requires a single call with an average latency of 13.8s, our base entity verification involves three calls averaging 44.0s. The mutation refinement step adds two calls averaging 28.1s; however, it is triggered conditionally, executing only when the first stage flags a potential hallucination. By limiting this additional cost to ambiguous cases, the design maintains a low average latency, rendering HALLU-DET highly practical for real-world applications.

📖 **Answer to RQ1-3:** HALLU-DET demonstrates robust detection accuracy (>95%) across all hallucination categories, while maintaining acceptable computational overhead.

6.3 Hallucination Mitigation Effectiveness (Study II)

6.3.1 *RQ2-1: Overall Mitigation Performance.* To answer RQ2-1, we evaluate HALLU-SHIELD's ability to not only reduce hallucinations but also improve overall summary quality.

Hallucination Rates. We report hallucination rates measured by HALLU-DET, which provides reliable annotations. Although HALLU-DET also supplies labels for training HALLU-VALUE, the training and evaluation datasets are fully disjoint, ensuring that our mitigation evaluation remains independent of the training process. As shown in Table 3, HALLU-SHIELD consistently achieves the

Table 3. Hallucination rate across mitigation methods and base models on HALLU-EVAL. Percentages show the relative reduction for HALLU-SHIELD compared to the Greedy Decoding baseline.

CLM	Dataset	Greedy	Factuality	CoT	Best-of-N	HALLU-SHIELD
Qwen2.5-Coder-7B	HALLU-EVAL (Easy)	0.24	0.19	0.30	0.28	0.22 (↓8%)
	HALLU-EVAL (Hard)	0.99	1.00	0.92	1.00	0.96 (↓3%)
	HALLU-EVAL	0.62	0.60	0.61	0.64	0.59 (↓5%)
CodeLlama-7B	HALLU-EVAL (Easy)	0.63	0.56	0.55	0.54	0.52 (↓17%)
	HALLU-EVAL (Hard)	0.92	0.90	0.91	0.94	0.90 (↓2%)
	HALLU-EVAL	0.78	0.73	0.73	0.74	0.71 (↓9%)
DeepSeek-Coder-6.7B	HALLU-EVAL (Easy)	0.38	0.44	0.33	0.43	0.28 (↓26%)
	HALLU-EVAL (Hard)	0.94	0.98	0.94	0.94	0.90 (↓4%)
	HALLU-EVAL	0.66	0.71	0.64	0.69	0.59 (↓11%)
Llama3-8B	HALLU-EVAL (Easy)	0.24	0.18	0.44	0.18	0.21 (↓13%)
	HALLU-EVAL (Hard)	0.83	0.83	0.96	0.92	0.74 (↓11%)
	HALLU-EVAL	0.54	0.51	0.70	0.55	0.48 (↓11%)

lowest overall hallucination rate across all four base models. It systematically outperforms the strong BoN baseline, our most competitive baseline (e.g., achieving a 0.59 vs. 0.69 hallucination rate on DeepSeek-Coder-6.7B and 0.48 vs. 0.55 on Llama3-8B). While prompting-based baselines (Factuality and CoT) occasionally achieve competitive results on the simpler HALLU-EVAL (Easy) subset, their performance is inconsistent and degrades significantly on the more challenging HALLU-EVAL (Hard) variants. On HALLU-EVAL (Hard), HALLU-SHIELD surpasses all baselines on three models, with particularly strong gains on DeepSeek-Coder-6.7B (0.90 vs. 0.94 for BoN) and Llama3-8B (0.74 vs. 0.92 for BoN). These results indicate that while simple prompting may yield marginal improvements on simpler cases, HALLU-SHIELD’s value-guided, sentence-level decisions provide more robust and reliable mitigation, especially for difficult scenarios.

An independent evaluation using the LLM Direct judge confirms consistent effectiveness: on HALLU-EVAL (Easy), HALLU-SHIELD reduces the average hallucination rate from 31.25% to 23.50%, while baselines often exacerbate hallucinations (e.g., CoT: 41.50% and Best-of-N at 35.50%). This validates that our value-guided search fundamentally improves semantic fidelity. Furthermore, HALLU-SHIELD generalizes well to unseen general-purpose models like Llama3-8B, reducing its hallucination rate from 29% to 24%. Even on the challenging adversarially perturbed set (HALLU-EVAL Hard), HALLU-SHIELD strictly maintains or lowers the near-100% baseline. This independent validation proves that our value-guided search fundamentally improves semantic fidelity rather than merely gaming the training metric.

Summary Quality. A critical question for any hallucination mitigation strategy is whether reducing hallucinations comes at the cost of overall summary quality. To gain a deep and comprehensive understanding on quality of code summary generated by HALLU-SHIELD, we employ a majority vote ensemble LLM-as-a-judge evaluation [61] to mitigate potential biases from any single model. In this setup, an ensemble of three diverse, state-of-the-art LLMs (GPT-5.1 [33], Qwen3.5-Plus [49], and DeepSeek-V3.2 [6]) performs side-by-side comparisons between summaries generated by HALLU-SHIELD and those generated by greedy decoding. Following the prompt from [61], the judges are specifically instructed to determine which summary is better along three key dimensions: Accuracy, Completeness, and Fluency, and the final decision is aggregated via majority voting. We use *win rate* to denote the percentage of evaluated examples for which the summary generated by HALLU-SHIELD is preferred over that generated by greedy decoding by the majority of judges.

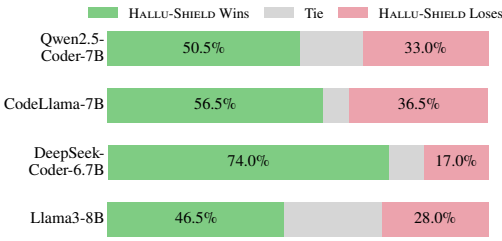


Fig. 9. Majority vote win rate of HALLU-SHIELD over greedy decoding.

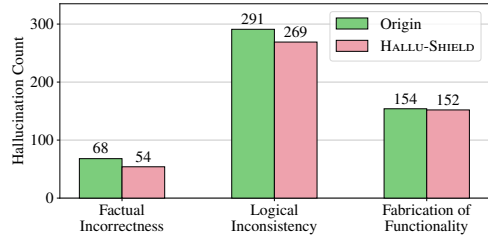


Fig. 10. Mitigation effect of HALLU-SHIELD.

The results, visualized in Figure 9, reveal a strong and consistent preference for HALLU-SHIELD. For example, HALLU-SHIELD achieves a decisive win rate of 74.0% for DeepSeek-Coder-6.7B and 56.5% for CodeLlama-7B, maintaining a strong positive margin even on the general-purpose Llama3-8B (46.5% wins vs. 28.0% losses). Furthermore, the variance among the individual judges within the ensemble is exceptionally low (e.g., win-rate variance ranges from 0.002 to 0.005), confirming that this strong preference is highly stable and free from single-model bias. To ensure evaluation reliability, we conduct a blinded human cross-validation on 100 samples. The human win rate (56.5%) closely aligns with the ensemble judge (58.5%). This agreement confirms that HALLU-SHIELD does not trade quality for factuality and faithfulness; rather, it genuinely improves overall quality.

Answer to RQ2-1: HALLU-SHIELD is highly effective, achieving lower hallucination rates than all baselines while also improving the overall quality of the generated summaries.

6.3.2 RQ2-2: Robustness Across Hallucination Categories. To understand where HALLU-SHIELD's mitigation capabilities are most impactful, we analyze its effect on the absolute count of each hallucination type before and after its application. Figure 10 provides a detailed breakdown of this mitigation effect. The results show that HALLU-SHIELD is most effective at reducing the two most common error types. It achieves a marked reduction in both factual incorrectness, lowering the count from 68 to 54 instances (a 20.6% reduction), and logical inconsistency, with a reduction from 291 to 269 instances. This demonstrates that our value-guided approach successfully steers generation away from paths that lead to factually and logically flawed statements. Conversely, HALLU-SHIELD has a minimal impact on fabrication of functionality (reducing from 154 to 152 instances). We attribute this to the influence of the model's flawed internal prior: when an LLM fundamentally hallucinates an entire concept, all generated candidate trajectories in the sentence-level search space tend to propagate this fabrication, leaving the value model with no faithful option to select. This suggests that such global inventions are less amenable to correction via inference-time decoding search alone.

Answer to RQ2-2: HALLU-SHIELD effectively mitigates factual incorrectness and logical inconsistency, but is less effective for fabrication of functionality.

6.3.3 RQ2-3: Ablation and Efficiency Analysis. To answer RQ2-3, we analyze the key drivers of HALLU-SHIELD's effectiveness and its mitigation efficiency trade-offs.

Ablation Study. To isolate the key drivers of HALLU-SHIELD's effectiveness, we conduct an ablation study analyzing its two core design principles: the specialized value model (HALLU-VALUE) and the proactive, step-by-step guidance. Figure 11 summarizes the ablation across base models. First, we find that a naive step-by-step search guided by the Original Qwen2.5-Coder-7B (i.e., Foundational

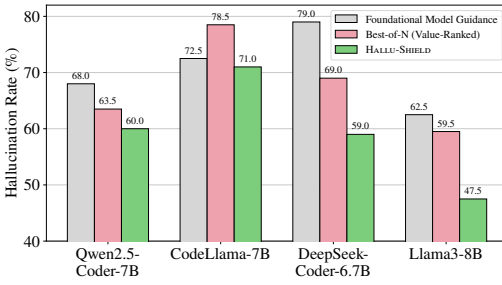


Fig. 11. Ablation study.

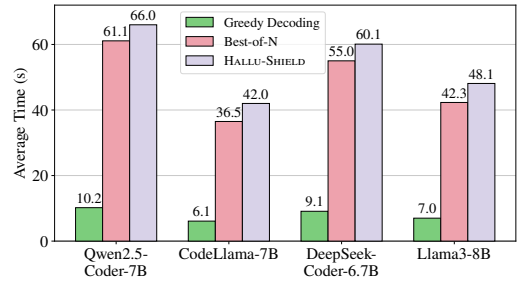


Fig. 12. Average time per complete summary.

Model Guidance) is insufficient and can even increase hallucinations compared to greedy decoding, confirming the necessity of a specialized value model. Next, while using HALLU-VALUE to rank completed summaries (Best-of-N) generally reduces hallucinations, demonstrating its effectiveness as a ranker, our full HALLU-SHIELD strategy significantly outperforms it. For instance, on Llama3-8B, HALLU-SHIELD reduces the hallucination rate to 47.5%, a substantial improvement over the 59.5% achieved by Best-of-N. This superiority validates the importance of HALLU-SHIELD’s proactive, step-by-step guidance. Unlike the reactive Best-of-N approach, which can only choose the “least flawed” summary from a set of completed candidates, HALLU-SHIELD intervenes during generation to prune paths likely to lead to hallucinations, addressing the root cause of hallucinations.

Mitigation Efficiency. To demonstrate the practical superiority of our approach, we further compare the computational cost of the various mitigation methods, measuring the average time in seconds required to generate a single summary, with the results summarized in Figure 12. This time measurement reflects the end-to-end cost, including candidate generation, external ranking for BoN, and HALLU-VALUE inference for HALLU-SHIELD, all run on identical hardware and batch settings. The results reveal that, as expected, all search-based methods significantly increase computational cost compared to the simple Greedy Decoding baseline, a necessary trade-off for improved factual accuracy. Among these advanced methods, HALLU-SHIELD’s time consumption is consistently on par with, and only slightly higher than, that of the strong Best-of-N baseline across all models; for instance, on DeepSeek-Coder-6.7B, HALLU-SHIELD requires 60.1 seconds, a marginal increase over BoN’s 55.0 seconds. However, while the computational costs are comparable, the performance benefits established in Table 3 are not: for similar computational budget, HALLU-SHIELD consistently achieves a lower hallucination rate. This highlights HALLU-SHIELD’s superior efficiency, as its proactive, sentence-level value guidance steers generation more effectively than BoN, retrospective ranking of multiple fully completed summaries.

Answer to RQ2-3: The effectiveness of HALLU-SHIELD is driven by the synergy between its specialized value model and its proactive search. Compared to BoN, HALLU-SHIELD offers better performance at a competitive computational cost.

6.4 Qualitative Analysis

We provide a qualitative walkthrough to illustrate the step-by-step mechanisms of our detection and mitigation methods. We use the deleteSnapshot method as a running case study to demonstrate how HALLU-DET identifies a subtle but critical hallucination, and how HALLU-SHIELD proactively prevents it from being generated.

As illustrated in Figure 13, the process begins with an LLM-generated summary for the deleteSnapshot method. Our HALLU-DET approach first applies direct entity-level detection. It parses

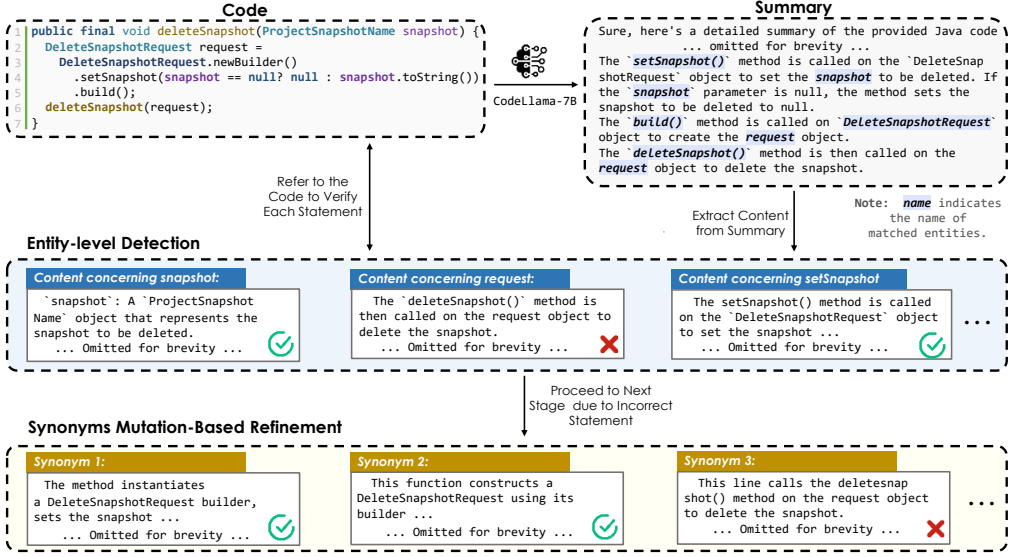


Fig. 13. Case study: HALLU-DET detection workflow on deleteSnapshot.

the source code and extracts key entities (e.g., the snapshot parameter, the request object, and setSnapshot() method). Concurrently, it analyzes the summary. While most statements are correctly verified, a critical hallucination is identified in the statement: “The deleteSnapshot() method is then called on the request object to delete the snapshot.” The initial detection step correctly flags this as potentially “Hallucinated”. This flagged segment is then passed to the synonymous mutation-based refinement process for a more robust check. This process generates 6 semantically equivalent mutations for re-evaluation. As shown, some synonyms (e.g., Synonym 1 and 2) which correctly rephrase the relationship are verified as “Correct”. However, Synonym 3, which repeats the original hallucination (“...called on the request object...”), is correctly identified as “Hallucinated”. Because at least one mutation is deemed hallucination, the refinement process confirms the initial finding, and the original statement is definitively classified as a hallucination.

This detected hallucination highlights a scenario where our mitigation method HALLU-SHIELD becomes crucial. As shown in Figure 14, let’s assume the base LLM, during generation, has already produced the correct part of the summary and is now at a state where it needs to generate the next sentence. The LLM might propose several candidate sentences. One candidate might be the previously identified hallucinatory statement: “The deleteSnapshot() method is then called on the request object...”. Our trained value model HALLU-VALUE, which has learned to associate such common but incorrect API usage patterns with low rewards, assigns a low long-term value (i.e., 2.65) to this candidate. In contrast, another candidate might be the factually correct statement: “The deleteSnapshot method is called with request object as an argument.” The HALLU-VALUE, recognizing that this phrasing accurately reflects the code semantics, assigns it a high long-term value (i.e., 3.27). The HALLU-SHIELD approach then selects this high-value candidate to append to the summary.

7 Threats to Validity

Generalizability across Models and Languages. Our empirical evaluation focuses on four open-source models and is limited to Java. Therefore, our quantitative findings may not generalize directly to all LLMs, particularly larger, closed-source models which may exhibit different behaviors, or to other programming languages with distinct characteristics such as Python. However, we

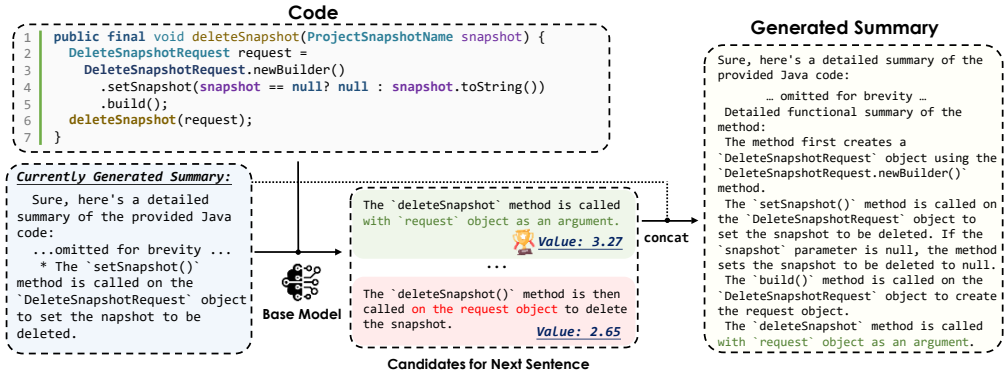


Fig. 14. Case study: HALLU-SHIELD mitigation with sentence-level value guidance.

believe the core principles of our methods, namely the synergistic process of entity-level detection and synonyms mutation-based refinement in HALLU-DET, as well as the far-sighted, value-guided search in HALLU-SHIELD, are general paradigms. Future works should validate this potential across a broader spectrum of models and languages.

The Limitation Inherited from the Original Dataset. As previously stated, our dataset originates from the code-to-text subset of CodeXGLUE [25], and as such, shares its inherent limitations. A notable constraint is the dataset's focus on function-level, relatively simple code snippets. These may not fully represent the complexity of large-scale, multi-file software projects, and future works should explore hallucination phenomena in more extensive code contexts.

Pathological Code in Adversarial Evaluation. Although predicate negation may produce control flows that are less common in practice, HALLU-EVAL mitigates overfitting to such adversarial cases by including the complementary HALLU-EVAL (Easy) subset of original code, thereby preserving both dataset diversity and practical relevance.

8 Conclusion

In this paper, we conduct a systematic study to unveil, detect, and mitigate hallucinations in LLM-based code summarization. We first construct HALLU-EVAL, a novel dataset designed to systematically induce and evaluate hallucinations in LLM-based code summarization. We then introduce HALLU-DET, which synergizes direct entity-level detection with synonymous mutation-based refinement to reliably identify both explicit and ambiguous hallucinations. Building on this, we further propose a value-guided inference-time mitigation strategy, HALLU-SHIELD, which employs a learned HALLU-VALUE to guide LLMs towards generating more accurate and faithful summaries without costly retraining. Our extensive experiments demonstrate that this systematic approach enables rigorous evaluation, accurate detection, and effective mitigation of hallucinations in LLM-based code summarization.

9 Data Availability

All experimental data and source code used in this paper are available at <https://github.com/CGCL-codes/naturalcc/tree/main/examples/code-summarization-hallucination>.

Acknowledgments

This work is supported by the Major Program (JD) of Hubei Province (Grant No. 2023BAA024). We would like to thank all the anonymous reviewers for their insightful comments.

References

- [1] Emad Aghajani, Csaba Nagy, Mario Linares-Vásquez, Laura Moreno, Gabriele Bavota, Michele Lanza, and David C Shepherd. 2020. Software documentation: the practitioners' perspective. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE 2020)*. 590–601.
- [2] Toufique Ahmed and Premkumar Devanbu. 2022. Few-shot training llms for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering*. 177:1–177:5.
- [3] Chao Chen, Kai Liu, Ze Chen, Yi Gu, Yue Wu, Mingyuan Tao, Zhihang Fu, and Jieping Ye. 2024. INSIDE: LLMs' Internal States Retain the Power of Hallucination Detection. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.
- [4] YunSeok Choi and Jee-Hyong Lee. 2023. CodePrompt: Task-agnostic prefix tuning for program and language generation. In *Findings of the Association for Computational Linguistics: ACL 2023*. 5282–5297.
- [5] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. In *Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS 2017)*.
- [6] DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *CoRR* abs/2512.02556 (2025).
- [7] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. Chain-of-Verification Reduces Hallucination in Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 3563–3578.
- [8] Hanxing Ding, Liang Pang, Zihao Wei, Huawei Shen, and Xueqi Cheng. 2025. Rowen: Adaptive Retrieval-Augmented Generation for Hallucination Mitigation in LLMs. In *Proceedings of the 3rd International ACM SIGIR Conference on Information Retrieval in the Asia Pacific (SIGIR-AP 2025)*.
- [9] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *Proceedings of the 46th International Conference on Software Engineering (ICSE '24)*. ACM, 81:1–81:13.
- [10] Anne Edmundson, Brian Holtkamp, Emanuel Rivera, Matthew Finifter, Adrian Mettler, and David Wagner. 2013. An empirical study on the effectiveness of security code review. In *Proceedings of the International Symposium on Engineering Secure Software and Systems*. Springer, 197–212.
- [11] Chunrong Fang, Weisong Sun, Yuchen Chen, Xiao Chen, Zhao Wei, Qunjun Zhang, Yudu You, Bin Luo, Yang Liu, and Zhenyu Chen. 2024. Esale: Enhancing code-summary alignment learning for source code summarization. *IEEE Transactions on Software Engineering* 50, 8 (2024), 2077–2095.
- [12] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*.
- [13] Yuhan Fu, Ruobing Xie, Xingwu Sun, Zhanhui Kang, and Xirong Li. 2025. Mitigating Hallucination in Multimodal Large Language Model via Hallucination-targeted Direct Preference Optimization. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*. Association for Computational Linguistics, 16563–16577.
- [14] Gemini Team. 2023. Gemini: A Family of Highly Capable Multimodal Models. *CoRR* abs/2312.11805 (2023).
- [15] Mojtaba Mostafavi Ghahfarokhi, Alireza Asadi, Arash Asgari, Bardia Mohammadi, and Abbas Heydarnoori. 2025. Predicting the understandability of computational notebooks through code metrics analysis. *Empirical Software Engineering* 30, 4 (2025), 98.
- [16] Michael G'unther, Jackmin Ong, Isabelle Mohr, Alaeddine Abdesslem, Tanguy Abel, Mohammad Kalim Akram, Susana Guzman, Georgios Mastrapas, Saba Sturua, Bo Wang, Maximilian Werk, Nan Wang, and Han Xiao. 2023. Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents. *CoRR* abs/2310.19923 (2023).
- [17] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *CoRR* abs/2401.14196 (2024).
- [18] Sonia Haiduc, Jairo Aponte, and Adrian Marcus. 2010. Supporting program comprehension with source code summarization. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering—Volume 2*. 223–226.
- [19] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 220:1–220:79.
- [20] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems* 43, 2 (2025), 42:1–42:55.

- [21] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, An Yang, Rui Men, Fei Huang, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. *CoRR* abs/2409.12186 (2024).
- [22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rockt"aschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS 2020)*. 9459–9474.
- [23] Bolun Li, Zhihong Sun, Tao Huang, Hongyu Zhang, Yao Wan, Ge Li, Zhi Jin, and Chen Lyu. 2024. IRCoco: Immediate Rewards-Guided Deep Reinforcement Learning for Code Completion. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 182–203.
- [24] Llama Team. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024).
- [25] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1*.
- [26] Junyu Luo, Cao Xiao, and Fenglong Ma. 2024. Zero-Resource Hallucination Prevention for Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 3586–3602.
- [27] Kishan Maharaj, Vitobha Munigala, Srikanth G. Tamilselvam, Prince Kumar, Sayandeep Sen, Palani Kodeswaran, Abhijit Mishra, and Pushpak Bhattacharyya. 2025. ETF: An Entity Tracing Framework for Hallucination Detection in Code Summaries. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*. 30639–30652.
- [28] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*. Association for Computational Linguistics, 9004–9017.
- [29] Antonio Mastropaolo, Matteo Ciniselli, Massimiliano Di Penta, and Gabriele Bavota. 2024. Evaluating code summarization techniques: A new metric and an empirical characterization. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 218:1–218:13.
- [30] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On Faithfulness and Factuality in Abstractive Summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 1906–1919.
- [31] Sewon Min, Kalpesh Krishna, Xinxu Lyu, Mike Lewis, Wen-tau Yih, Pang Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FACTScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 12076–12100.
- [32] Feng Nan, Ramesh Nallapati, Zhiguo Wang, Cicero Nogueira dos Santos, Henghui Zhu, Dejiao Zhang, Kathleen McKeown, and Bing Xiang. 2021. Entity-Level Factual Consistency of Abstractive Text Summarization. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2021)*. 2727–2733.
- [33] OpenAI. 2025. GPT-5 Language Model. <https://chat.openai.com>.
- [34] Hadas Orgad, Michael Toket, Zorik Gekhman, Roi Reichart, Idan Szpektor, Hadas Koteck, and Yonatan Belinkov. 2025. LLMs Know More Than They Show: On the Intrinsic Representation of LLM Hallucinations. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*.
- [35] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS 2022)*. 27730–27744.
- [36] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*. Curran Associates, Inc.
- [37] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *CoRR* abs/2308.12950 (2023).

- [38] Satrio Adi Rukmono, Lina Ochoa, and Michel RV Chaudron. 2023. Achieving high-level software component summarization via hierarchical chain-of-thought prompting and static code analysis. In *Proceedings of the 2023 IEEE International Conference on Data and Software Engineering (ICoDSE)*. IEEE, 7–12.
- [39] Iman Saberi, Amirreza Esmaeili, Fatemeh Hendijani Fard, and Fuxiang Chen. 2025. Advfusion: Adapter-based knowledge transfer for code summarization on code language models. In *Proceedings of the 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 563–574.
- [40] Albert Sawczyn, Jakub Binkowski, Denis Janiak, Bogdan Gabrys, and Tomasz Kajdanowicz. 2026. FactSelfCheck: Fact-Level Black-Box Hallucination Detection for LLMs. In *Findings of the Association for Computational Linguistics: EACL 2026*. 5603–5621.
- [41] Eva Sharma, Luyang Huang, Zhe Hu, and Lu Wang. 2019. An Entity-Driven Framework for Abstractive Summarization. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP 2019)*. 3278–3289.
- [42] Ensheng Shi, Yanlin Wang, Lun Du, Junjie Chen, Shi Han, Hongyu Zhang, Dongmei Zhang, and Hongbin Sun. 2022. On the evaluation of neural code summarization. In *Proceedings of the 44th international conference on software engineering*. 1597–1608.
- [43] Chia-Yi Su and Collin McMillan. 2024. Distilled GPT for Source Code Summarization. *Automated Software Engineering* 31, 1 (2024), 22.
- [44] Weihang Su, Changyue Wang, Qingyao Ai, Yiran Hu, Zhijing Wu, Yujia Zhou, and Yiqun Liu. 2024. Unsupervised Real-Time Hallucination Detection based on the Internal States of Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*. 14379–14391.
- [45] Weisong Sun, Chunrong Fang, Yuchen Chen, Qunjun Zhang, Guanhong Tao, Yudu You, Tingxu Han, Yifei Ge, Yuling Hu, Bin Luo, and Zhenyu Chen. 2024. An extractive-and-abstractive framework for source code summarization. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 75:1–75:39.
- [46] Weisong Sun, Chunrong Fang, Yudu You, Yuchen Chen, Yi Liu, Chong Wang, Jian Zhang, Qunjun Zhang, Hanwei Qian, Wei Zhao, Yang Liu, and Zhenyu Chen. 2023. A Prompt Learning Framework for Source Code Summarization. *CoRR abs/2312.16066* (2023).
- [47] Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Qunjun Zhang, Hanwei Qian, Yang Liu, and Zhenyu Chen. 2023. Automatic Code Summarization via ChatGPT: How Far Are We? *CoRR abs/2305.12865* (2023).
- [48] Weisong Sun, Yun Miao, Yuekang Li, Hongyu Zhang, Chunrong Fang, Yi Liu, Gelei Deng, Yang Liu, and Zhenyu Chen. 2025. Source Code Summarization in the Era of Large Language Models. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE 2025)*. 1882–1894.
- [49] Qwen Team. 2026. Qwen3.5: Accelerating Productivity with Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>
- [50] Yuchen Tian, Weixiang Yan, Qian Yang, Xuandong Zhao, Qian Chen, Wen Wang, Ziyang Luo, Lei Ma, and Dawn Song. 2025. CodeHal: Investigating Code Hallucinations in LLMs via Execution-Based Verification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. AAAI Press, 25300–25308.
- [51] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. 2023. A Stitch in Time Saves Nine: Detecting and Mitigating Hallucinations of LLMs by Validating Low-Confidence Generation. *CoRR abs/2307.03987* (2023).
- [52] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip S. Yu. 2019. Multi-modal Attention Network Learning for Semantic Source Code Retrieval. In *Proceedings of the 34th ACM/IEEE International Conference on Automated Software Engineering (ASE 2019)*. 13–25.
- [53] Yao Wan, Shijie Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Dezhong Yao, Hai Jin, and Lichao Sun. 2022. You See What I Want You to See: Poisoning Vulnerabilities in Neural Code Search. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2022)*. 1233–1245.
- [54] Yao Wan, Wei Zhao, Hongyu Zhang, Yulei Sui, Guandong Xu, and Hai Jin. 2022. What Do They Capture? - A Structural Analysis of Pre-Trained Language Models for Source Code. In *Proceedings of the 44th ACM/IEEE International Conference on Software Engineering (ICSE 2022)*. 2377–2388.
- [55] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S. Yu. 2018. Improving Automatic Source Code Summarization via Deep Reinforcement Learning. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE 2018)*. 397–407.
- [56] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R Lyu. 2022. No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In *Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering*. 382–394.

- [57] Chong Wang, Jian Zhang, Yebo Feng, Tianlin Li, Weisong Sun, Yang Liu, and Xin Peng. 2025. Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation. *ACM Transactions on Software Engineering and Methodology* 34, 7 (2025), 191:1–191:27.
- [58] Xiyao Wang, Zhengyuan Yang, Linjie Li, Hongjin Lu, Yuancheng Xu, Chung-Ching Lin, Kevin Lin, Furong Huang, and Lijuan Wang. 2024. Scaling Inference-Time Search with Vision Value Model for Improved Visual Comprehension. *CoRR* abs/2412.03704 (2024).
- [59] Weibin Wu, Yuhang Cao, Ning Yi, Rongyi Ou, and Zibin Zheng. 2025. Detecting and Reducing the Factual Hallucinations of Large Language Models with Metamorphic Testing. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1432–1453.
- [60] Wenlong Wu, Haofen Wang, Bohan Li, Peixuan Huang, Xinzhe Zhao, and Lei Liang. 2025. MultiRAG: A Knowledge-Guided Framework for Mitigating Hallucination in Multi-Source Retrieval Augmented Generation. In *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE 2025)*. 3070–3083.
- [61] Yang Wu, Yao Wan, Zhaoyang Chu, Wenting Zhao, Ye Liu, Hongyu Zhang, Xuanhua Shi, and Philip S Yu. 2025. Can Large Language Models Serve as Evaluators for Code Summarization? *IEEE Transactions on Software Engineering* 51, 12 (2025), 3205–3217.
- [62] Wen Xiao and Giuseppe Carenini. 2023. Entity-based SpanCopy for Abstractive Summarization to Improve the Factual Consistency. In *Proceedings of the 4th Workshop on Computational Approaches to Discourse (CODI 2023)*. 70–81.
- [63] Borui Yang, Md Afif Al Mamun, Jie M. Zhang, and Gias Uddin. 2025. Hallucination Detection in Large Language Models with Metamorphic Relations. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 425–445.
- [64] Zhen Yang, Jacky Wai Keung, Xiao Yu, Yan Xiao, Zhi Jin, and Jingyu Zhang. 2023. On the significance of category prediction for code-comment synchronization. *ACM Transactions on Software Engineering and Methodology* 32, 2 (2023), 30:1–30:41.
- [65] Shangbo Yun, Shuhuai Lin, Xiaodong Gu, and Beijun Shen. 2024. Project-Specific Code Summarization with In-Context Learning. *Journal of Systems and Software* 216 (2024), 112149.
- [66] Haopeng Zhang, Semih Yavuz, Wojciech Kryscinski, Kazuma Hashimoto, and Yingbo Zhou. 2022. Improving the Faithfulness of Abstractive Summarization via Entity Coverage Control. In *Findings of the Association for Computational Linguistics: NAACL 2022*. 528–535.
- [67] Jiaxin Zhang, Zhuohang Li, Kamalika Das, Bradley Malin, and Sricharan Kumar. 2023. SAC3: Reliable Hallucination Detection in Black-Box Language Models via Semantic-aware Cross-check Consistency. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 15445–15458.

Received 2026-02-25; accepted 2026-03-24